

Rapport

Mode Pas à Pas et Contrôle de Vitesse

Sabir Anas

Introduction

Ce rapport technique présente les travaux réalisés sur l'implémentation des fonctionnalités de mode pas à pas et de contrôle de vitesse dans le cadre du développement du Jeu de la Vie. Le document détaille les choix techniques, les solutions implémentées et les résultats obtenus.

1 Séance de Méthode Agile

Dans le cadre du développement itératif du Jeu de la Vie, notre équipe a organisé un meeting dans la séance de méthode agile pour identifier et prioriser de nouvelles fonctionnalités à implémenter. Cette réunion a permis de mettre en lumière les besoins des utilisateurs et d'établir un backlog de fonctionnalités à développer.

1.1 Déroulement de meeting

La séance s'est tenue en format stand-up meeting, où chaque membre de l'équipe a pu proposer des fonctionnalités et argumenter leur utilité. Après une phase de brainstorming, nous avons utilisé la méthode de vote pondéré pour prioriser les différentes propositions.

1.2 Fonctionnalités proposées

Les principales fonctionnalités proposées lors de cette séance étaient :

1. Mode pas à pas (avancer/reculer dans la simulation)
2. Contrôle de vitesse de simulation
3. Zoom et dézoom sur la grille
4. Mode multijoueur avec cellules de différentes couleurs
5. Règles personnalisables
6. Configurations prédéfinies supplémentaires
7. Export/import de configurations
8. Statistiques d'évolution

J'ai choisi de travailler sur les deux fonctionnalités - le mode pas à pas et le contrôle de vitesse - .

2 Implémentation du Mode Pas à Pas

2.1 Analyse des besoins

Le mode pas à pas devait permettre aux utilisateurs de :

- Avancer d'une génération à la fois
- Revenir en arrière à des états précédents
- Visualiser l'évolution précise des cellules
- Comprendre les transitions d'états

2.2 Solution technique

J'ai utilisé un système d'historique pour enregistrer chaque état de la grille, permettant ainsi de naviguer dans le temps. Pour cela, ces variables d'instance ont été ajoutées à la classe principale :

```

1 // Pour la fonctionnalité pas pas
2 private List<Grille> historique = new ArrayList<>();
3 private int positionHistorique = -1;

```

Explication des variables :

- `historique` : Liste stockant toutes les versions (états) de la grille à chaque étape
- `positionHistorique` : Index indiquant la position actuelle dans l'historique
 - Valeur initiale à -1 (aucun état enregistré)
 - 0 pour le premier état, 1 pour le deuxième, etc.
 - Cette variable est essentielle pour permettre la navigation dans l'historique

2.3 Explication détaillée des méthodes

2.3.1 Méthode `ajouterHistorique()`

```

1 private void ajouterHistorique() {
2     // Crée une copie de la grille actuelle
3     Grille copie = copierGrille(grille);
4
5     // Si modification après un pas arrière, supprimer l'historique
    suivant
6     while (historique.size() > positionHistorique + 1) {
7         historique.remove(historique.size() - 1);
8     }
9
10    historique.add(copie);
11    positionHistorique = historique.size() - 1;
12 }

```

Objectif : Enregistrer l'état actuel de la grille dans l'historique et gérer les branches temporelles en cas de modification après navigation dans l'historique.

Explication détaillée :

- `Grille copie = copierGrille(grille);` : Crée une copie indépendante de l'état actuel de la grille
- La boucle `while` : Gère le cas où une modification intervient après navigation dans l'historique

```

1     while (historique.size() > positionHistorique + 1) {
2         historique.remove(historique.size() - 1);
3     }
4

```

Exemple :

- Avec 10 étapes dans l'historique (positions 0 à 9)
- Navigation à la position 5 (`positionHistorique = 5`)
- Modification de la grille
- La boucle supprime alors les positions 6 à 9, créant une nouvelle branche temporelle à partir de la position 5
- `historique.add(copie);` : Ajoute la copie de la grille à l'historique
- `positionHistorique = historique.size() - 1;` : Met à jour la position courante dans l'historique

2.3.2 Méthode `copierGrille(Grille source)`

```

1 private Grille copierGrille(Grille source) {
2     Grille copie = new Grille(source.getHauteur(), source.getLargeur(),
3     source.estTorrique());
4
5     for (int i = 0; i < source.getHauteur(); i++) {
6         for (int j = 0; j < source.getLargeur(); j++) {
7             copie.getCellule(i, j).setEtat(source.getCellule(i, j).
8             estVivant());
9         }
10    }
11    return copie;
12 }

```

Objectif : Créer une copie profonde indépendante de la grille source.

Explication :

- Création d'une nouvelle grille avec mêmes dimensions et propriétés
- Copie itérative de l'état de chaque cellule
- Retourne la nouvelle grille entièrement copiée

2.3.3 Méthode restaurerEtat(int index)

```

1 private void restaurerEtat(int index) {
2     if (index >= 0 && index < historique.size()) {
3         Grille etatPrecedent = historique.get(index);
4
5         // Copier l'etat precedent dans la grille actuelle
6         for (int i = 0; i < grille.getHauteur(); i++) {
7             for (int j = 0; j < grille.getLargeur(); j++) {
8                 grille.getCellule(i, j).setEtat(etatPrecedent.getCellule
9                 (i, j).estVivant());
10            }
11        }
12        actualiserGrille();
13    }
14 }

```

Objectif : Restaurer un état précédent de la grille à partir de l'historique.

Explication :

- Vérification de la validité de l'index
- Récupération de l'état précédent dans l'historique
- Copie itérative des états des cellules
- Mise à jour de l'affichage

2.3.4 Code des boutons pour le mode pas à pas

```

1 pasAvant = new JButton("Pas Avant");
2 pasAvant.addActionListener(e -> {
3     if (!enCours) {
4         prochaineEtape();
5         ajouterHistorique();
6         actualiserMenu();
7     }
8 });

```

```

9
10 pasArriere = new JButton("Pas Arri re");
11 pasArriere.addActionListener(e -> {
12     if (!enCours && positionHistorique > 0) {
13         positionHistorique--;
14         restaurerEtat(positionHistorique);
15         actualisermenu();
16     }
17 });
18 pasArriere.setEnabled(false);

```

Objectif : Implémenter les contrôles d'interface pour la navigation pas à pas.

Explication :

- Bouton "Pas Avant" :
 - Calcule l'étape suivante
 - Ajoute l'état à l'historique
 - Met à jour les contrôles
 - Désactivé pendant les simulations automatiques
- Bouton "Pas Arrière" :
 - Décrémente la position dans l'historique
 - Restaure l'état précédent
 - Initialement désactivé (pas d'historique disponible)

2.3.5 Méthode actualisermenu()

```

1 private void actualiserControles() {
2     demarrer.setEnabled(!enCours);
3     arreter.setEnabled(enCours);
4     torique.setEnabled(!enCours);
5     pasAvant.setEnabled(!enCours);
6     pasArriere.setEnabled(!enCours && positionHistorique > 0);
7     reset.setEnabled(!enCours);
8 }

```

Objectif : Gérer l'état de menu de l'interface utilisateur.

Explication :

- Active/désactive les boutons selon l'état de la simulation
- Gère les dépendances entre les différents contrôles
- Assure la cohérence de l'interface

3 Implémentation du Contrôle de Vitesse

3.1 Analyse des besoins

Le contrôle de vitesse devait permettre aux utilisateurs de :

- Ralentir la simulation pour observer des évolutions complexes
- Accélérer la simulation pour atteindre rapidement un état stable
- Ajuster la vitesse en temps réel, même pendant que la simulation est en cours

3.2 Solution technique

3.2.1 Variables et constantes

```

1 // Valeurs pour le contr le de vitesse
2 private static final int DELAI_MIN = 50; // 50ms = rapide
3 private static final int DELAI_MAX = 1000; // 1000ms = lent
4 private static final int DELAI_INIT = 200; // valeur initiale
5
6 private int delaiActuel = DELAI_INIT; // D lai actuel

```

Explication :

- Définition des limites de vitesse (50ms à 1000ms)
- Valeur initiale à 200ms
- Variable stockant le délai courant

3.2.2 Interface utilisateur

```

1 // Panel pour le contr le de vitesse
2 JPanel vitessePanel = new JPanel();
3 vitessePanel.setLayout(new BorderLayout(vitessePanel, BorderLayout.Y_AXIS));
4 vitessePanel.setBorder(BorderFactory.createTitledBorder("Vitesse"));
5
6 // Label pour afficher la vitesse actuelle
7 vitesseLabel = new JLabel("Vitesse: " + delaiActuel + "ms");
8 vitesseLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
9
10 // Slider pour ajuster la vitesse
11 vitesseSlider = new JSlider(JSlider.HORIZONTAL, DELAI_MIN, DELAI_MAX,
12     DELAI_INIT);
13 vitesseSlider.setInverted(true); // Inverser pour que gauche=lent,
14     droite=rapide
15 vitesseSlider.setMajorTickSpacing(200);
16 vitesseSlider.setMinorTickSpacing(50);
17 vitesseSlider.setPaintTicks(true);
18 vitesseSlider.setPaintLabels(true);
19 vitesseSlider.setAlignmentX(Component.CENTER_ALIGNMENT);

```

Explication :

- Création d'un panneau dédié au contrôle de vitesse
- Label affichant la valeur courante
- Slider avec :
 - Valeurs de 50 à 1000ms Inversion pour une logique intuitive Graduations principales et secondaires Affichage des valeurs

3.2.3 Gestion des événements

```

1 // couteur pour le slider
2 vitesseSlider.addChangeListener(new ChangeListener() {
3     public void stateChanged(ChangeEvent e) {
4         delaiActuel = vitesseSlider.getValue();
5         vitesseLabel.setText("Vitesse: " + delaiActuel + "ms");
6
7         // Si la simulation est en cours, mettre jour le timer
8         if (enCours) {
9             timer.stop();
10            timer.setDelay(delaiActuel);
11            timer.start();
12        } else {

```

```

13         // Sinon, juste mettre à jour le délai du timer
14         timer.setDelay(delaiActuel);
15     }
16 }
17 });

```

Explication :

- Met à jour la variable `delaiActuel` avec la valeur du slider
- Actualise l’affichage du label
- Si simulation en cours :
 - Arrête temporairement le timer
 - Modifie le délai
 - Redémarre le timer
- Sinon, met simplement à jour le délai pour le prochain démarrage

3.2.4 Initialisation du timer

```

1 // lance prochaineEtape toutes les <delaiActuel> ms
2 timer = new Timer(delaiActuel, e -> {
3     prochaineEtape();
4     ajouterHistorique();
5 });

```

Explication :

- Crée un timer avec le délai initial
- À chaque déclenchement :
 - Calcule l’étape suivante
 - Ajoute l’état à l’historique

4 Intégration et Tests

4.1 Stratégie d’intégration

Les fonctionnalités ont été développées et intégrées selon l’approche suivante :

1. Développement indépendant du mode pas à pas
2. Développement indépendant du contrôle de vitesse
3. Tests unitaires de chaque composant
4. Intégration progressive
5. Tests d’intégration globaux

4.2 Tests utilisateurs

Des tests utilisateurs ont permis de valider :

- **Mode pas à pas :**
 - Navigation correcte dans les états
 - Cohérence de l’historique
 - Gestion des modifications après retour arrière
- **Contrôle de vitesse :**
 - Réactivité des ajustements
 - Fluidité aux différentes vitesses
 - Affichage correct de la vitesse courante

5 Difficultés rencontrées et solutions

5.1 Mode pas à pas

- **Problème** : Gestion des modifications après retour arrière
 - **Solution** : Suppression des états "futurs" et création d'une nouvelle branche
- **Problème** : Consommation mémoire de l'historique
 - **Solution** : Limitation implicite par la suppression des branches

5.2 Contrôle de vitesse

- **Problème** : Réactivité des changements en cours de simulation
 - **Solution** : Arrêt/redémarrage du timer
- **Problème** : Plage de valeurs appropriée
 - **Solution** : Tests itératifs pour déterminer 50-1000ms

6 Conclusion

L'implémentation de ces fonctionnalités a apporté des améliorations significatives :

- Meilleur contrôle de la simulation
- Expérience utilisateur enrichie
- Outil pédagogique plus efficace

L'approche itérative et les tests utilisateurs ont permis d'obtenir des résultats satisfaisants tout en identifiant des pistes d'amélioration pour les versions futures.